

Snowflake Tasks, Sequences & Auto-Increment – Detailed Notes

1. Snowflake Tasks – Overview

What is a Task?

A **Snowflake Task** is used to **schedule SQL execution** inside Snowflake.

♦ A task can run:

- INSERT
- MERGE
- COPY
- UPDATE
- Stored Procedures

♦ Important Limitation

One task can execute only ONE SQL statement

How to run multiple SQL statements?

✓ **Option 1:** Create a **Task Tree** (parent → child → grandchild)

✓ **Option 2:** Put all SQL statements inside a **Stored Procedure** and execute it using a task

2. Types of Tasks

- **Snowflake-Managed Task (Serverless Task)**
- **User-Managed Task**

A. Snowflake-Managed Task (Serverless Task)

- No warehouse required
- Snowflake automatically manages compute
- You may specify **initial warehouse size**, but **not warehouse name**

Example

```
CREATE TASK serverless_task
```

```
SCHEDULE = '5 MINUTE'
```

```
AS
```

```
INSERT INTO employees VALUES (1, 'John');
```

✓ Best for lightweight jobs

✓ No warehouse cost management

B. User-Managed Task

- Requires an **explicit warehouse**
- User controls compute resources

Example

```
CREATE TASK user_managed_task
```

```
WAREHOUSE = my_wh
```

```
SCHEDULE = '5 MINUTE'
```

```
AS
```

```
INSERT INTO employees VALUES (2, 'David');
```

- ✓ Best for heavy workloads
- ✓ Full control over warehouse size

Snowflake-Managed vs User-Managed Task

Feature	Snowflake-Managed	User-Managed
Warehouse required	✗ No	✓ Yes
Compute management	Snowflake	User
Cost control	Automatic	Manual
Best for	Small / scheduled jobs	Heavy workloads

3. Task Timeout

What is Task Timeout?

Task Timeout defines the maximum amount of time a Snowflake task is allowed to run before Snowflake automatically terminates it.

This prevents:

- Infinite loops
- Long-running or stuck tasks
- Uncontrolled compute usage

Default Task Timeout

60 minutes (1 hour)

This applies to **each individual task**, not the entire task tree.

Even in a task tree (root → child → grandchild), **every task has its own 60-minute limit**.

What Happens When Timeout Is Reached?

If a task exceeds the timeout:

1. Snowflake automatically stops the task
2. Task status becomes FAILED
3. Error message shows timeout exceeded
4. Dependent (child) tasks do NOT run

Task Timeout Parameter

You can change the timeout using:

USER_TASK_TIMEOUT_MS

- Unit: milliseconds
- Applies per task

Example – Set Timeout to 30 Minutes

```
ALTER TASK employee_root_task
```

```
SET USER_TASK_TIMEOUT_MS = 1800000;
```

✚ 30 minutes = $30 \times 60 \times 1000 = 1,800,000$ ms

Example – Increase Timeout for Heavy Job (2 Hours)

```
ALTER TASK large_load_task
```

```
SET USER_TASK_TIMEOUT_MS = 7200000;
```

Task Timeout in a Task Tree (Important Concept)

Consider this task tree:

Root Task → Child Task → Grandchild Task

Scenario Example

- Root task runs for **5 minutes**
- Child task runs for **70 minutes**

👉 Result:

- Root task → **SUCCESS**
- Child task → **FAILED (timeout)**
- Grandchild task → **NOT EXECUTED**

Relation Between Timeout and Scheduling

If a task:

- Is **still running** and
- Its **next scheduled time** arrives

That scheduled run is **SKIPPED by default, not queued**

This is **separate from timeout behavior**

Common Interview Trap Question

Q: If a task runs longer than its schedule interval, will Snowflake stop it?

A: No. Snowflake stops a task only when timeout is reached, not when the next schedule arrives.

4. Task Scheduling Rules

What Are Task Scheduling Rules?

Task scheduling rules define when a task runs, when it skips execution, and how Snowflake handles overlaps when a task takes longer than expected.

These rules are critical for data consistency, cost control, and avoiding duplicate loads.

Rule 1: Only the Root Task Has a Schedule

- In a task tree, only the root task can have a SCHEDULE
- Child and grandchild tasks must NOT have schedules
- They use the AFTER keyword to define dependency

Example

```
CREATE TASK root_task
WAREHOUSE = my_wh
SCHEDULE = '5 MINUTE'
AS
INSERT INTO t1 VALUES (1);
```

NEXT

```
CREATE TASK child_task
WAREHOUSE = my_wh
AFTER root_task
AS INSERT INTO t2 SELECT * FROM t1;
```

Rule 2: If a Task Is Still Running, the Next Scheduled Run Is Skipped

By default, Snowflake does not allow overlapping executions.

Scenario

- Task scheduled every 4 minutes
- Task execution time = 6 minutes

👉 Result:

- Next scheduled run is SKIPPED
- Task runs again at the next interval

✓ No queuing

✓ No parallel execution by default

Rule 3: Tasks Are Not Queued

Snowflake tasks:

- ✗ Do not queue missed runs
- ✗ Do not retry automatically

If a run is skipped:

That execution is lost permanently

Rule 4: Overlapping Execution Is Disabled by Default

- Only **one instance of a task** can run at a time
- Prevents:
 - Duplicate inserts
 - Data corruption
 - Race conditions

Enable Overlapping (Optional)

ALTER TASK root_task

SET ALLOW_OVERLAPPING_EXECUTION = TRUE;

Use **only when logic is idempotent**

Rule 5: Child Tasks Ignore the Schedule Interval

Child task execution:

- Depends only on parent completion
- Can take more time than the root task schedule

Example

- Root task scheduled every 2 minutes
- Child task runtime = 3 minutes

Child task still runs successfully

It starts after root finishes, not based on schedule

Rule 6: Task Timeout Is Independent of Schedule

- Default timeout = **60 minutes**
- A task is **NOT stopped** just because the next schedule arrives
- Task stops **only when timeout is exceeded**

Rule 7: Task Execution Order in a Tree

Execution always follows:

Root → Child → Grandchild

✓ Child starts only after parent **success**

✗ Child does NOT run if parent fails or times out

Rule 8: Newly Created Tasks Are Suspended

- Tasks **do not run automatically** after creation
- Must be explicitly resumed

ALTER TASK root_task RESUME;

Rule 9: Task Tree Modification Requires Root Task to Be Suspended

- Add child tasks
- Add grandchild tasks
- Modify dependencies

Root task must be suspended first

ALTER TASK root_task SUSPEND;

Rule 10: Daylight Saving Time (DST) Impact

For schedules between **2:00 AM – 3:00 AM**:

- Tasks may **run twice**
- Or **not run at all**

✓ Best Practice:

- Use **UTC**
- Avoid critical jobs in DST windows

Key Behaviors

- If a task is **still running**, the **next scheduled run is skipped**
- Snowflake prevents **overlapping executions by default**

Example

- Task scheduled every **4 minutes**
- Task execution time = **6 minutes**

👉 Next scheduled run **SKIPPED**

Real-World Scheduling Scenario

Example

- Root task: every 5 minutes
- Child task: runs 8 minutes
- Overlapping: disabled (default)

Behavior

- Root run #1 → child starts
- Root run #2 → skipped
- Root run #3 → runs normally

5. Allow Overlapping Execution

What Is Allow Overlapping Execution?

Allow Overlapping Execution is a task setting that controls whether multiple instances of the same task are allowed to run at the same time when the task is scheduled frequently.

◆ **Default behavior:**

Overlapping execution is **NOT allowed**

◆ **When enabled:**

Snowflake allows a new task run to start **even if the previous run has not finished**

Default Behavior (Overlapping Disabled)

Scenario

- Task schedule: **every 2 minutes**
- Task execution time: **5 minutes**

Result (Default)

- Run at 00:00 → starts
- Run at 00:02 → **✗** skipped
- Run at 00:04 → **✗** skipped
- Run at 00:06 → starts (after previous completes)

✓ Prevents duplicate processing

✓ Avoids data corruption

Enabling Allow Overlapping Execution

SQL Command

```
ALTER TASK root_task
```

```
SET ALLOW_OVERLAPPING_EXECUTION = TRUE;
```

This setting is applied **only at the root task level**

Behavior When Overlapping Is Enabled

Scenario

- Schedule: every 2 minutes
- Execution time: 5 minutes

Result

- 00:00 → Run #1 starts
- 00:02 → Run #2 starts
- 00:04 → Run #3 starts

✓ Multiple instances run **in parallel**

Safe Use Case (Idempotent Logic)

Example – MERGE (Safe)

MERGE INTO target t

USING source s

ON t.id = s.id

WHEN MATCHED THEN UPDATE SET t.name = s.name

WHEN NOT MATCHED THEN INSERT VALUES (s.id, s.name);

- ✓ Same data processed multiple times
- ✓ No duplication

Dangerous Use Case (Non-Idempotent)

Example – INSERT (Unsafe)

INSERT INTO sales

SELECT * FROM staging_sales;

- ✗ Multiple overlapping runs → duplicate rows

Real Interview Scenario

Q: Root task runs every 2 minutes, child task takes 4 minutes. Will child task run?

A: ✓ Yes. Child execution depends on root completion, not schedule interval.

Q: If overlapping is enabled, can two child tasks run in parallel?

A: ✓ Yes, if triggered by overlapping root executions.

Best Practices

- ✓ Enable overlapping **only when necessary**
- ✓ Use **MERGE** instead of INSERT
- ✓ Track execution using:

INFORMATION_SCHEMA.TASK_HISTORY

- ✓ Monitor cost impact carefully

6. Task Initial State

What Is the Task Initial State?

The task initial state refers to the status of a task immediately after it is created in Snowflake.

By default, every newly created task is in a *SUSPENDED* state.

After Creating a Task

```
CREATE TASK daily_load_task
```

```
WAREHOUSE = my_wh
```

```
SCHEDULE = '10 MINUTE'
```

```
AS
```

```
INSERT INTO sales SELECT * FROM staging_sales;
```

Result:

- Task exists
- Task **does NOT** run
- Task status = **SUSPENDED**

This is an intentional Snowflake design to:

- Prevent accidental execution
- Allow validation before running
- Avoid unexpected compute usage

Why Are Tasks Created as Suspended?

- ✓ Safety
- ✓ Cost control
- ✓ Dependency setup before execution
- ✓ Verification of logic

Especially important for task trees, where dependencies must be defined correctly first.

How to Check Task State

```
SHOW TASKS;
```

or

```
SELECT name, state
```

```
FROM INFORMATION_SCHEMA.TASKS;
```

Possible states:

- SUSPENDED
- STARTED (running / enabled)

How to Start (Resume) a Task

Resume a Task

```
ALTER TASK daily_load_task RESUME;
```

Once resumed:

- Task starts executing as per schedule
- Child tasks can now be triggered

How to Stop (Suspend) a Task

Suspend a Task

```
ALTER TASK daily_load_task SUSPEND;
```

Effects:

- Task stops running
- Future scheduled runs are skipped
- Entire task tree halts if root task is suspended

Important Rule in Task Trees

Resume Order Matters

👉 Always resume tasks from bottom to top

Correct Order

1st ALTER TASK child_task RESUME;

2nd ALTER TASK root_task RESUME;

Suspending a Root Task

```
ALTER TASK root_task SUSPEND;
```

✦ Result:

- Entire task tree stops running
- Child tasks cannot execute even if resumed

Fast Way to Resume Entire Task Tree

Instead of resuming one by one:

```
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('root_task_name');
```

- ✓ Resumes root + all dependent tasks at once
- ✓ Best for large task trees

7. Task Tree (Running Multiple SQL Statements)

What Is a Task Tree?

A Task Tree in Snowflake is a hierarchical structure of tasks that allows you to execute multiple SQL statements sequentially, even though each individual task can run only one SQL statement.

Tasks are connected using dependencies, forming:

Root Task → Child Task → Grandchild Task

Why Do We Need a Task Tree?

Snowflake Limitation

One task = One SQL statement

To run multiple SQL statements:

- ✓ Chain tasks using dependencies
- ✓ Execute them in a defined order

This chained structure is called a Task Tree.

Core Components of a Task Tree

1. Root Task

- Has a **SCHEDULE**
- Triggers the entire tree
- Only one root task per tree

2. Child Task

- Has **NO** schedule
- Uses (AFTER) keyword
- Runs only after parent task completes successfully

Basic Task Tree Example

Step 1: Create 2 Tables

```
CREATE TABLE employees (  
  emp_id INT,  
  emp_name STRING  
);
```

```
CREATE TABLE employees_two (  
    emp_id INT,  
    emp_name STRING  
);
```

Step 2: Create Root Task

```
CREATE TASK employee_root_task  
    WAREHOUSE = my_wh  
    SCHEDULE = '5 MINUTE'  
AS  
INSERT INTO employees VALUES (1, 'Alice');
```

Root task inserts data into **employees table**.

Step 3: Create Child Task

```
CREATE TASK employee_child_task  
    WAREHOUSE = my_wh  
    AFTER employee_root_task  
AS  
INSERT INTO employees_two  
SELECT * FROM employees;
```

Child task runs **only after root task finishes**.

Managing Large Task Trees - Enable Entire Tree at Once

```
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('employee_root_task');
```

✓ Enables all dependent tasks automatically

Execution Flow

1. Root task starts (based on schedule)
2. Root task completes successfully
3. Child task starts automatically
4. If root fails → child does **NOT** run

8. Root Task vs Child Task

Root Task – Definition

A Root Task is the top-level task in a task tree that controls the schedule and triggers the execution of all dependent tasks.

Only one root task exists per task tree

Child Task – Definition

A Child Task is a dependent task that does not have its own schedule and executes only after its parent task completes successfully.

Child tasks rely on the AFTER dependency, not on time-based scheduling.

Root Task vs Child Task

Feature	Root Task	Child Task
Schedule	✓ Required	✗ Not allowed
Trigger	Time-based (cron / minutes)	Dependency-based
Dependency keyword	✗ Not used	✓ AFTER
Controls task tree	✓ Yes	✗ No
Can exist alone	✓ Yes	✗ No
Runs independently	✓ Yes	✗ Only after parent
Overlapping execution setting	✓ Configurable	✗ Inherited

Root Task Example

```
CREATE TASK employee_root_task
```

```
WAREHOUSE = my_wh
```

```
SCHEDULE = '5 MINUTE'
```

```
AS
```

```
INSERT INTO employees VALUES (1, 'Alice');
```

This task:

- Has a schedule
- Starts the task tree
- Runs every 5 minutes

Child Task Example

```
CREATE TASK employee_child_task
```

```
WAREHOUSE = my_wh
```

```
AFTER employee_root_task
```

```
AS
```

```
INSERT INTO employees_two
```

```
SELECT * FROM employees;
```

This task:

- Has no schedule
- Runs only after employee_root_task completes
- Cannot run independently

Scheduling vs Dependency (Important Concept)

What is Scheduling?

Scheduling means a task runs based on time, not based on another task.

In Snowflake:

- Only ROOT TASKS are scheduled
- Schedule can be:
 - Every n minutes
 - Cron expression

Example (Scheduling)

```
CREATE TASK root_task
```

```
WAREHOUSE = my_wh
```

```
SCHEDULE = '2 MINUTE'
```

```
AS
```

```
INSERT INTO t1 VALUES (1);
```

This task runs every 2 minutes, regardless of other tasks.

What is Dependency?

Dependency means a task runs only after another task completes successfully.

In Snowflake:

- Child tasks are dependency-based
- Dependency is defined using the AFTER keyword

Example (Dependency)

```
CREATE TASK child_task  
  WAREHOUSE = my_wh  
  AFTER root_task  
AS  
INSERT INTO t2 SELECT * FROM t1;
```

This task:

- Has no schedule
- Runs only after root_task finishes successfully

Scenario 1 Reminder– Overlapping execution: ❌ Disabled

- Root schedule: every 2 minutes
- Root runtime: 1 minute
- Child runtime: 4 minutes
- Grandchild runtime: 2 minutes
- Overlapping: ❌ Disabled

Execution Timeline

00:00 → Root starts
00:01 → Root completes → Child starts
00:02 → Root starts again
00:03 → Root completes (child still running)
00:05 → Child completes → Grandchild starts
00:06 → Root starts again
00:07 → Root completes → Child starts
00:08 → Grandchild completes
00:11 → Child completes → Grandchild starts
00:13 → Grandchild completes

Scenario 2 Reminder – Overlapping execution: ☒ ENABLED

- Root schedule: every 2 minutes
- Root runtime: 1 minute
- Child runtime: 4 minutes
- Grandchild runtime: 2 minutes
- Overlapping execution: ☒ ENABLED

Execution Timeline

00:00 → Root starts
00:01 → Root completes → Child starts
00:02 → Root starts again
00:03 → Root completes (Child still running)
00:04 → Root scheduled
 → Root is NOT running
 → Root starts (NO SKIP)
00:05 → Child completes → Grandchild starts
00:06 → Root starts again
00:07 → Root completes → Child starts
00:08 → Grandchild completes
00:10 → Root scheduled
 → Root is NOT running
 → Root starts (NO SKIP)
00:11 → Child completes → Grandchild starts
00:13 → Grandchild completes

Important Clarification (Why No Skip?)

- Root is **never running at its next schedule time**
- Even if it were running, **overlapping is enabled**
- So **Root NEVER skips**
- Child and Grandchild **wait only for their parent run**, not for other parallel runs

How to Read This (Very Important)

- Root starts only on schedule
- Child starts only when Root completes
- Grandchild starts only when Child completes
- Schedule does not control child or grandchild execution

9. Task Tree Limits

- Max child tasks per task → 100
- Max tasks per tree → 1000
- Only root task has a schedule
- Root must be suspended to change task tree

Root Task

└─ Child Task 1

| └─ Grandchild Task 1

| └─ Grandchild Task 2

└─ Child Task 2

└─ Child Task 3

10. Resuming Tasks – Correct Order

Rule

Resume from bottom → top

Why?

Because dependencies must be enabled first.

✓ Correct Order (BOTTOM → TOP)

ALTER TASK grandchild_task RESUME; -- ✓

ALTER TASK child_task RESUME; -- ✓

ALTER TASK root_task RESUME; -- ✓

✓ All tasks move to **STARTED** state correctly.

11. Enable Entire Task Tree at Once

Initial State (Default)

Root_Task → SUSPENDED

Child_Task → SUSPENDED

Grandchild_Task → SUSPENDED

✓ Enable Entire Tree (One Command)

```
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('Root_Task');
```

Resulting State

Grandchild_Task → STARTED

Child_Task → STARTED

Root_Task → STARTED

✓ Order handled automatically by Snowflake

✓ Enables **root + all children + grandchildren**

✓ Best for large task trees

12. Finding Task Dependencies

A. Direct Children Only

```
SELECT *  
FROM TABLE(INFORMATION_SCHEMA.TASK_DEPENDENTS(  
    TASK_NAME => 'employee_root_task',  
    RECURSIVE => FALSE  
));
```

B. Full Tree (Recursive)

```
SELECT *  
FROM TABLE(INFORMATION_SCHEMA.TASK_DEPENDENTS(  
    TASK_NAME => 'employee_root_task',  
    RECURSIVE => TRUE  
));
```

Parameter	Description
TASK_NAME	The name of the task you want to check dependents for
RECURSIVE	TRUE → returns all dependents (direct + indirect) FALSE → returns only direct children

13. Viewing Task Graph (UI)

- 1) **Login to Snowflake Web Interface**
- 2) Navigate to **Databases → Your Database → Your Schema**
- 3) Click **Tasks**
- 4) Select a **root task** or any task you want to investigate
- 5) Click on **Graph / Lineage View** (usually called “Task Graph”)

- ✓ Visual parent-child relationship
- ✓ Helps debugging complex trees

14. Task Run History

Task run history is crucial for **monitoring, troubleshooting, and auditing** your Snowflake tasks. It lets you see **when tasks ran, their status, and any errors**.

Viewing Task Run History via SQL

Snowflake stores task execution info in **INFORMATION_SCHEMA.TASK_HISTORY**.

```
SELECT *  
FROM INFORMATION_SCHEMA.TASK_HISTORY  
WHERE TASK_NAME = 'root_task_name'  
ORDER BY SCHEDULED_TIME DESC;
```

Example

Suppose you have a **task tree**:

Query:

```
SELECT TASK_NAME, SCHEDULED_TIME, START_TIME, END_TIME, STATE  
FROM INFORMATION_SCHEMA.TASK_HISTORY  
WHERE TASK_NAME IN ('Root_Task', 'Child_Task', 'Grandchild_Task')  
ORDER BY SCHEDULED_TIME;
```

Sample Output:

TASK_NAME	SCHEDULED_TIME	START_TIME	END_TIME	STATE
Root_Task	2026-01-20 00:00	00:00	00:01	SUCCEEDED

TASK_NAME	SCHEDULED_TIME	START_TIME	END_TIME	STATE
Child_Task	2026-01-20 00:01	00:01	00:05	SUCCEEDED
Grandchild_Task	2026-01-20 00:05	00:05	00:07	SUCCEEDED
Root_Task	2026-01-20 00:02	00:02	00:03	SUCCEEDED
Child_Task	2026-01-20 00:03	00:03	00:07	SUCCEEDED
Grandchild_Task	2026-01-20 00:07	00:07	00:09	SUCCEEDED

Example Timeline from Task History

00:00 → Root_Task scheduled & started
 00:01 → Root_Task completes → Child_Task starts
 00:05 → Child_Task completes → Grandchild_Task starts
 00:07 → Grandchild_Task completes
 00:02 → Root_Task scheduled again & started
 00:03 → Root_Task completes → Child_Task starts
 00:07 → Child_Task completes → Grandchild_Task starts
 00:09 → Grandchild_Task completes

Understanding Task History

STATE Column

STATE	Meaning
SUCCEEDED	Task ran successfully
FAILED	Task failed; check ERROR_CODE & ERROR_MESSAGE
SKIPPED	Task skipped because previous run still running (if overlapping disabled)
ABORTED	Task manually stopped during execution

SCHEDULED_TIME vs START_TIME

- SCHEDULED_TIME = planned start time
- START_TIME = actual start time (may differ if overlapping is disabled)

15. Manual Task Execution

In Snowflake, tasks are **normally scheduled** or run automatically based on **dependencies**. However, you can also **manually execute a task** at any time. This is extremely useful for:

- Testing task logic
- Catching up on missed runs
- Triggering child tasks after fixing an error

SQL Command to Execute a Task Manually

```
EXECUTE TASK task_name;
```

How Manual Execution Works

1. The task runs **immediately**, regardless of its schedule.
2. If it has **dependent tasks**, they will run **after the parent completes**, just like normal scheduled execution.
3. Overlapping rules apply:
 - If overlapping is **enabled**, task can start even if it's already running
 - If overlapping is **disabled**, manual execution is skipped if the task is running

Example Task Tree

```
EXECUTE TASK Root_Task;
```

Resulting Timeline

Root_Task starts immediately

Root_Task completes → Child_Task starts

Child_Task completes → Grandchild_Task starts

Grandchild_Task completes

Matches the **normal dependency flow**, but triggered manually instead of by schedule

16. External Dependencies Limitation

Snowflake tasks are designed to run SQL statements inside Snowflake based on schedules or dependencies. They cannot directly depend on external events, such as files arriving in S3, data landing in Azure Blob, or completion of external ETL jobs.

Understanding this limitation is crucial for designing task trees and orchestration workflows.

Tasks run only inside Snowflake

- Supported statements: INSERT, UPDATE, MERGE, COPY INTO, calling stored procedures
- Cannot wait for external files, events, or system signals

External dependencies must be handled outside Snowflake

- Use orchestration tools like:
 - Airflow
 - Control-M
 - dbt
- These tools can check for files in S3, run Python scripts, or trigger Snowflake stored procedures when conditions are met

17. External Orchestration Tools

Snowflake tasks cannot schedule external events → use orchestration tools

Airflow and **Control-M** are commonly used for external scheduling

Orchestration tools can trigger Snowflake tasks/stored procedures after external conditions

How It Works

Scenario: Load a file from S3 → Transform → Insert into Snowflake

Task 1: Check S3 for file

Task 2: If file exists, call Snowflake stored procedure or task

Task 3: Transform data in Snowflake

Task 4: Load data into final table

Snowflake task **handles only internal SQL logic**

Orchestration tool **handles scheduling, dependencies, and external triggers**

Example: Task tree in Snowflake runs after external orchestration tool triggers it

18. Daylight Saving Time Issue

Snowflake tasks run based on time zone of the schedule (UTC by default if not specified)

When local time moves forward (spring) → 1 hour is skipped

- Tasks scheduled during the missing hour may not run

When local time moves backward (fall) → 1 hour repeats

- Tasks scheduled during the repeated hour may run twice

Example: Task Scheduled Every Hour at 2:30 AM

Scenario 1 – Spring Forward (DST starts)

- Time jumps from 2:00 → 3:00 AM
- Task scheduled at 2:30 AM → **does NOT run**
- Next run is at 3:30 AM

Scenario 2 – Fall Back (DST ends)

- Time repeats 2:00 → 3:00 AM
- Task scheduled at 2:30 AM → **may run twice**
- Next run continues normally

Recommendation

- ✓ Avoid critical jobs during DST windows
- ✓ Use UTC where possible

19. Sequences in Snowflake

What is a Sequence?

A **sequence** in Snowflake is an object that generates **unique numeric values**, typically used to populate **primary key columns** or **auto-incrementing IDs**.

Key Points About Sequences

- Sequences generate **incremental numbers**
- They are **independent objects** in a schema
- Can be used in **INSERT statements** or **default column values**
- Can be **shared across tables** if needed

Create Sequence

```
CREATE SEQUENCE employee_seq  
  
START WITH 1  
  
INCREMENT BY 1  
  
MINVALUE 1  
  
MAXVALUE 1000000  
  
CYCLE = FALSE;
```

Explanation:

Clause	Description
START WITH	First value the sequence will generate
INCREMENT BY	Step between numbers (default = 1)
MINVALUE	Minimum allowed value
MAXVALUE	Maximum allowed value
CYCLE	If TRUE → restarts after reaching MAXVALUE

Using a Sequence in an INSERT Statement

```
INSERT INTO employees (employee_id, name)  
VALUES (employee_seq.NEXTVAL, 'Ramesh');
```

- employee_seq.NEXTVAL → gets the **next number from the sequence**
- Each call generates a **unique value**

Setting a Sequence as Default Value for a Column

```
CREATE TABLE employees (  
    employee_id NUMBER DEFAULT employee_seq.NEXTVAL,  
    name STRING DEFAULT 'Unknown'  
);
```

- When inserting without specifying employee_id, Snowflake automatically uses **next sequence value**
- Useful for **auto-increment behavior**

20. Auto-Increment / Identity Column

Purpose

Automatically generate unique IDs on insert

Example Table

```
CREATE TABLE employees (  
    emp_id INT AUTOINCREMENT,  
    emp_name STRING DEFAULT 'Romesh'  
);
```

Explanation:

Clause	Description
NUMBER	Data type of the column
AUTOINCREMENT	Automatically generates sequential numbers starting from 1

Insert Without ID

```
INSERT INTO employees (name) VALUES ('Romesh');  
INSERT INTO employees (name) VALUES ('Alice');  
INSERT INTO employees (name) VALUES ('Bob');
```

Resulting Table:

employee_id	name
1	Romesh
2	Alice
3	Bob

- ✓ emp_id generated automatically, no need to specify it.
- ✓ emp_name defaults if not provided

Insert With Explicit ID (Optional)

```
INSERT INTO employees (employee_id, name) VALUES (100, 'David');
```

- Snowflake **accepts your value** if explicitly provided
- Subsequent auto-increment continues from **the highest generated value**, not the manually inserted one

21. Constraints (Mentioned for SQL Sessions)

Types of Constraints

Constraint Type	Description
NOT NULL	Ensures column cannot have NULL values
PRIMARY KEY	Uniquely identifies each row in a table (combines NOT NULL + UNIQUE)
UNIQUE	Ensures values in a column or combination of columns are unique
CHECK	Ensures column values satisfy a specific condition
DEFAULT	Provides a default value when none is supplied on insert

Example Table With Constraints

```
CREATE TABLE employees (  
    employee_id NUMBER AUTOINCREMENT(1000,1),      -- Identity column  
    name STRING DEFAULT 'Unknown' NOT NULL,        -- Default + NOT NULL  
    email STRING UNIQUE,                          -- Must be unique  
    age NUMBER CHECK (age >= 18),                  -- Must be 18 or older  
    department STRING,  
    PRIMARY KEY (employee_id)                     -- Primary key constraint  
);
```

Explanation:

Column	Constraint Applied	Effect
employee_id	PRIMARY KEY	Unique and identifies each row
name	NOT NULL + DEFAULT	Cannot be null; defaults to 'Unknown' if missing
email	UNIQUE	No duplicate emails allowed
age	CHECK	Only values >= 18 allowed
department	None	Optional column

Notes on Snowflake Constraints

1. **NOT NULL:** Enforced strictly
2. **PRIMARY KEY / UNIQUE / CHECK:**
 - **By default, Snowflake does not enforce these**
 - Mainly used for **documentation and optimization**
 - Enforcement can be enabled in **Enterprise+ editions** using clustering or unique enforcement
3. **DEFAULT:** Automatically fills in missing values

22. Interview-Critical Summary

Topic	Key Point
Task runs	One SQL only
Root task	Has schedule
Child task	Uses AFTER
Default timeout	60 mins
Overlap	Disabled by default
Task state	Suspended on creation
Enable tree	SYSTEM\$TASK_DEPENDENTS_ENABLE
External dependency	Not supported
Sequence	Independent object
Identity column	Table-level auto-increment